

核心工作流

使用 Kimi API 的流式输出功能

📄 复制页面



Kimi 大模型在收到用户提出的问题后，会先进行推理、再**逐个 Token 生成回答**，在我们前两个章节的例子中，我们都选择等待 Kimi 大模型将所有 Tokens 生成完毕后，再打印（print）Kimi 大模型回复的内容，这通常要花费数秒的时间。如果你的问题足够复杂，且 Kimi 大模型生成的回复长度足够长，完整等待模型生成结果的时间可能会被拉长到 10 秒甚至 20 秒，这会极大降低用户的使用体验。为了改善这种情况，并及时给予用户反馈，我们提供了流式输出的能力，即 Streaming，我们将讲解 Streaming 的原理，并结合实际的代码来说明：

- 如何使用流式输出；
- 使用流式输出时的常见问题；
- 在不使用 Python SDK 的场合下如何处理流式输出；

如何使用流式输出

流式输出（Streaming），一言以蔽之，就是每当 Kimi 大模型生成了一定数量的 Tokens 时（通常情况下，这个数量是 1 Token），立刻将这些 Tokens 传输给客户端，而不再是等待所有 Tokens 生成完毕后再传输给客户端。当你与 **Kimi 智能助手** 进行对话时，Kimi 智能助手的回复是按字符逐个“跳”出来的，这即是流式输出的表现之一，**流式输出能让用户第一时间看到 Kimi 大模型输出的第一个 Token，减少用户的等待时间。**

你可以通过这样的方式（stream=True）来使用流式输出，并获得流式输出的响应：

`python` `node.js`

KIMI 开放平台



```
api_key = "MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
base_url = "https://api.moonshot.cn/v1",
)
```

```
stream = client.chat.completions.create(
    model = "kimi-k2.6",
    messages = [
        {"role": "system", "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你"},
        {"role": "user", "content": "你好，我叫李雷，1+1等于多少?"},
    ],
    stream=True, # <-- 注意这里，我们通过设置 stream=True 开启流式输出模式
)
```

当启用流式输出模式 (stream=True)，SDK 返回的内容也发生了变化，我们不再直接访问返回值中的 choices
而是通过 for 循环逐个访问返回值中每个单独的块 (chunk)

```
for chunk in stream:
    # 在这里，每个 chunk 的结构都与之前的 completion 相似，但 message 字段被替换成了 delta
    delta = chunk.choices[0].delta # <-- message 字段被替换成了 delta 字段

    if delta.content:
        # 我们在打印内容时，由于是流式输出，为了保证句子的连贯性，我们不人为地添加
        # 换行符，因此通过设置 end="" 来取消 print 自带的换行符。
        print(delta.content, end="")
```

使用流式输出时的常见问题

当您成功运行上述代码，并了解了流式输出的基本原理后，现在让我们向您讲述一些流式输出的细节和常见问题，以便于你更好的实现自己的业务逻辑。

接口细节

type: application/json) 的响应, 而是使用 Content-type: text/event-stream (简称

KIMI 开源平台 支持服务端源源不断地向客户端传输数据，在使用 Kimi 大模型的场景，可以理解为服务端源源不断地向客户端传输 Tokens。

当你查看 **SSE** 的 HTTP 响应体时，它看起来像这样：

```
data: {"id":"cml-1305b94c570f447fbde3180560736287","object":"chat.completion.chunk"}
data: {"id":"cml-1305b94c570f447fbde3180560736287","object":"chat.completion.chunk"}
...
data: {"id":"cml-1305b94c570f447fbde3180560736287","object":"chat.completion.chunk"}
data: {"id":"cml-1305b94c570f447fbde3180560736287","object":"chat.completion.chunk"}
data: [DONE]
```

在 **SSE** 的响应体中，我们约定数据块均以 **data:** 为前缀，紧跟一个合法的 JSON 对象，随后以两个换行符 **\n\n** 结束当前传输的数据块。最后，在所有数据块均传输完成时，会使用 **data:** **[DONE]** 来标识传输已完成，此时可断开网络连接。

Tokens 计算

当使用流式输出模式时，有两种计算 Tokens 的方式，最直接也是最准确的一种计算 Tokens 的方式，是等待所有数据块传输完毕后，通过访问最后一个数据块中的 `usage` 字段来查看整个流式输出过程中产生的 `prompt_tokens / completion_tokens / total_tokens`。

```
...  
  
data: {"id":"cml-1305b94c570f447fbde3180560736287","object":"chat.completion.chunk"  
                                             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
                                             通过访问最后一个数据块中的 usage 字段来查  
  
data: [DONE]
```

 Kimi K2.6 模型已正式发布，长程代码编写能力更强更稳。

络连接中断，客户端程序错误等)，此时，往往最后一个数据块尚未传输完毕，也就无从得知整个请求消耗的 Tokens 数量。为了避免这种计算 Tokens 失败的场景，我们建议将每个已经获取的数据块的内容保存下来，并在请求结束后（无论是否成功结束），使用 Tokens 计算接口计算已经产生的总消耗量，示例代码如下所示：

python **node.js**

KIMI 开放平台 `import httpx`
`import OpenAI`



```
client = OpenAI(
    api_key = "MOONSHOT_API_KEY", # 在这里将 MOONSHOT_API_KEY 替换为你从 Kimi 开放平台
    base_url = "https://api.moonshot.cn/v1",
)

stream = client.chat.completions.create(
    model = "kimi-k2.6",
    messages = [
        {"role": "system", "content": "你是 Kimi，由 Moonshot AI 提供的人工智能助手，你"},
        {"role": "user", "content": "你好，我叫李雷，1+1等于多少?"}
    ],
    stream=True, # <-- 注意这里，我们通过设置 stream=True 开启流式输出模式
)

def estimate_token_count(input: str) -> int:
    """
    在这里实现你的 Tokens 计算逻辑，或是直接调用我们的 Tokens 计算接口计算 Tokens

    https://api.moonshot.cn/v1/tokenizers/estimate-token-count
    """
    header = {
        "Authorization": f"Bearer {os.environ['MOONSHOT_API_KEY']}",
    }
    data = {
        "model": "kimi-k2.6",
        "messages": [
            {"role": "user", "content": input},
        ]
    }
    r = httpx.post("https://api.moonshot.cn/v1/tokenizers/estimate-token-count",
    r.raise_for_status()
    return r.json()["data"]["total_tokens"]

completion = []
```

KIMI 开放平台



```
if delta.content:
    completion.append(delta.content)

>

print("completion_tokens:", estimate_token_count("".join(completion)))
```

如何终止输出

如果你想要终止流式输出，你可以直接关闭 HTTP 网络连接，或是直接丢弃后续的数据块。例如：

```
for chunk in stream:
    if condition:
        break
```

在不使用 SDK 的情况下如何处理流式输出

如果你不想使用 Python SDK 来处理流式输出，而是想直接以对接 HTTP 接口的方式来使用 Kimi 大模型（例如某些没有 SDK 的语言，或是你有自己独特的业务逻辑而 SDK 无法满足的情况），我们给出一些示例来帮助你理解如何正确处理 HTTP 中 SSE 响应体（在这里我们仍然以 Python 代码为例，详细的说明将以注释的形式呈现）。

python node.js

```
data = {
    "model": "kimi-k2.6",
    "messages": [
        # 具体的 messages
    ],
    "stream": True,
}

# 使用 httpx 向 Kimi 大模型发出 chat 请求，并获得响应 r
r = httpx.post("https://api.moonshot.cn/v1/chat/completions", json=data)
if r.status_code != 200:
    raise Exception(r.text)

data: str

# 在这里，我们使用了 iter_lines 方法来逐行读取响应体
for line in r.iter_lines():
    # 去除每一行收尾的空格，以便更好地处理数据块
    line = line.strip()

    # 接下来我们要处理三种不同的情况：
    # 1. 如果当前行是空行，则表明前一个数据块已接收完毕（即前文提到的，通过两个换行符结束数据块）
    # 2. 如果当前行为非空行，且以 data: 开头，则表明这是一个数据块传输的开始，我们去除 data: 前缀
    # 3. 如果当前行为非空行，但不以 data: 开头，则表明当前行仍然归属上一个正在传输的数据块，

    if len(line) == 0:
        chunk = json.loads(data)

        # 这里的处理逻辑可以替换成你的业务逻辑，打印仅是为了展示处理流程
        choice = chunk["choices"][0]
        usage = choice.get("usage")
        if usage:
            print("total_tokens:", usage["total_tokens"])
        delta = choice["delta"]
        role = delta.get("role")
```

KIMI

开放平台

```

content = delta.get("content")
if content:
    print(content, end="")
>
data = "" # 重置 data
elif line.startswith("data: "):
    data = line.lstrip("data: ")

# 当数据块内容为 [DONE] 时，则表明所有数据块已发送完毕，可断开网络连接
if data == "[DONE]":
    break
else:
    data = data + "\n" + line # 我们仍然在追加内容时，为其添加一个换行符，因为这可能是

```

以上是以 Python 为例的流式输出处理流程，如果你使用其他语言，也可以正确处理流式输出的内容，其基本步骤如下：

1. 发起 HTTP 请求，并在请求体中，将 `stream` 参数设置为 `true`；
2. 接收服务端返回的响应，注意到响应 `Headers` 中的 `Content-Type` 为 `text/event-stream`，则说明当前响应内容为流式输出；
3. 逐行读取响应内容并解析数据块（数据块以 JSON 格式呈现），注意通过 `data:` 前缀及换行符 `\n` 来判断数据块的开始位置和结束位置；
4. 通过判断当前数据块内容是否为 `[DONE]` 来判断是否已传输完成；

注意，请始终使用 `data: [DONE]` 来判断数据是否已传输完成，而不是使用 `finish_reason` 或其他方式。如果未接收到 `data: [DONE]` 的消息块，即使已经获取了 `finish_reason=stop` 的信息，也不应视作数据块传输已完成。换句话说，在未接收到 `data: [DONE]` 的数据块前，都应视作消息是不完整的。

在流式输出过程中，只有 `content` 字段会被流式输出，即每个数据块包含 `content` 的部分 Tokens，而对于不需要流式输出的字段，例如 `role` 和 `usage`，我们通常会在第一个或最后一个数据块中一次呈现，而不会在每个数据块中都包含 `role` 和 `usage` 字段（具体的，`role` 字

后一个数据块中出现，而在前面的数据块中不会包含 `usage` 字段）。

KIMI 开放平台



n>1 时如何处理

在某些场合，我们会希望输出多个结果以供选择，此时正确的做法是将请求参数中的 `n` 设置为比 1 大的一个值。在流式输出中，我们同样支持 `n>1` 的使用方式，在这种场合下，我们需要添加一些额外的代码来判断当前数据块的 `index` 值，来确定传输的数据块具体归属于第几个回复，我们用示例代码来说明：

`python` `node.js`

KIMI 开放平台

```
data = {
    "model": "kimi-k2.6",
    "messages": [
        # 具体的 messages
    ],
    "stream": True,
    "n": 2, # <-- 注意这里，我们要求 Kimi 大模型输出 2 个回复
}

# 使用 httpx 向 Kimi 大模型发出 chat 请求，并获得响应 r
r = httpx.post("https://api.moonshot.cn/v1/chat/completions", json=data)
if r.status_code != 200:
    raise Exception(r.text)

data: str

# 在这里，我们预先构建一个 List，用于存放不同的回复消息，由于我们设置了 n=2，因此我们将 List 初始化为
messages = [{}, {}]

# 在这里，我们使用了 iter_lines 方法来逐行读取响应体
for line in r.iter_lines():
    line = line.strip()

    if len(line) == 0:
        chunk = json.loads(data)

        for choice in chunk["choices"]:
            index = choice["index"]
            message = messages[index]
            usage = choice.get("usage")
            if usage:
                message["usage"] = usage
            delta = choice["delta"]
            role = delta.get("role")
            if role:
```

KIMI

开放平台

if content:

message["content"] = message["content"] + content



```
>data = "" # 重置 data
elif line.startswith("data: "):
    data = line.lstrip("data: ")
```

```
if data == "[DONE]":
    break
```

```
else:
    data = data + "\n" + line
```

在组装完所有 messages 后，我们分别打印其内容

```
for index, message in enumerate(messages):
    print("index:", index)
    print("message:", json.dumps(message, ensure_ascii=False))
```

当 `n>1` 时，处理流式输出的要点在于，你需要先根据数据块的 `index` 值来判断当前数据块的内容归属于第几个回复消息，再进行后续的逻辑处理。

此页面对您有帮助吗？

 是

 否

JSON Mode >

